

Project: ULK Academic Activity and Resource Management System

Course: Advanced Database Management

Lecturer: **KAMATE KATENDE TIMOTHEE**

Email: kamatekatende@gmail.com / ktimothée@ulk.ac.rw

Phone: +250786554365

1. Background

Kigali Independent University manages academic activities across multiple faculties. However, academic data such as students, lecturers, courses, classrooms, laboratories, research projects, and events is currently scattered across different systems.

You are required to design and implement a centralized PostgreSQL database system that integrates all academic operations into one structured system.

2. Project Objective

You are required to design, model, implement, populate, and analyze a PostgreSQL database system that supports:

Student management, lecturer management, course management, enrollment, classroom and laboratory allocation, academic supervision, research activities, academic events, and resource management.

3. Project Rules (Important)

- You must design the system from real academic understanding, not assumptions copied from internet templates.
- Every design decision must be justifiable using academic logic.
- All diagrams and database structures must be consistent across all phases.
- You must use PostgreSQL object-relational features where required.

4. Phase 1: Requirements Analysis

4.1 Data Collection

You must collect real-world information using interviews or observation.

You must interview at least:

- 2 lecturers
- 3 students

You must include in your report:

- Full names of all interviewees
- Contact details (email or phone number)
- Date of interview
- Summary of responses (what they explained, not yes/no answers)

This section must be included in an appendix for verification.

4.2 Functional Requirements (What the system must do)

The system must support the following core operations:

- Student registration and profile management
- Lecturer registration and assignment
- Course creation and allocation to departments
- Student enrollment in courses
- Scheduling and allocation of classrooms and laboratories
- Recording and managing attendance or participation in academic activities
- Research project supervision and tracking
- Academic event creation and participation management
- Resource tracking across departments

4.3 Non-Functional Requirements (System qualities)

The system must:

- Maintain data integrity across all relationships
- Support efficient query execution for large datasets
- Allow concurrent access without data corruption

- Ensure scalability for future academic expansion
- Maintain security and controlled access to data
- Ensure consistency between related academic entities

4.4 Assumptions

You must clearly state any assumptions made during system design. Examples include academic structure, course delivery rules, and resource usage rules.

5. Phase 2: Conceptual Design (ER Model)

5.1 Entities

You must identify at least 10 entities.

Core entities include:

Student

Lecturer

Course

Department

Faculty

ResearchProject

Classroom

Laboratory

AcademicEvent

Enrollment

You may add more entities if needed, but they must be justified.

5.2 Relationships

You must define at least 12 relationships.

Each relationship must represent a real academic interaction.

Examples:

Student enrolls in Course

Lecturer teaches Course

Department offers Course

Student belongs to Department
Course is scheduled in Classroom
Laboratory is used for practical sessions
Lecturer supervises ResearchProject
Student participates in AcademicEvent
Lecturer organizes AcademicEvent
Student attends Course sessions
Course requires Classroom allocation
Lecturer assigned to Department

6. Phase 3: Enhanced ER Modeling (EER)

You must extend your ER model using advanced concepts.

6.1 Specialization

You must define at least one hierarchy.

Example:

Person $\rightarrow$ Student, Lecturer

You must clearly define:

- inherited attributes
- role-specific attributes

6.2 Generalization

You must combine similar entities into a parent entity.

Example:

Classroom + Laboratory $\rightarrow$ AcademicSpace

6.3 Constraints

You must define and justify:

- Whether specialization is disjoint or overlapping
- Whether it is total or partial

Justification must be based on real academic logic.

6.4 Aggregation

You must model at least one aggregation.

Example:

Lecturer supervises Student ResearchProject

This supervision is treated as a higher-level relationship for tracking academic mentorship.

6.5 Category (Union Type)

You must define at least one category.

Example:

Student + Lecturer $\rightarrow$ EventParticipant

Event participants may be either students or lecturers.

7. Phase 4: PostgreSQL Implementation

You must implement your design using PostgreSQL.

Required Features

Feature	Requirement
User Defined Type	e.g. Address
Composite Type	e.g. ContactInformation
Array Attribute	e.g. PhoneNumbers[]
JSONB Attribute	e.g. Research metadata
Inheritance	Person $\rightarrow$ Student, Lecturer

Each feature must be used in a real table (not isolated examples).

8. Phase 5: Data Population

You must populate the database with realistic data.

Minimum requirements:

- Student: 100 records
- Lecturer: 20 records
- Course: 30 records
- Other tables: meaningful data

Total dataset must be at least 300 records.

9. Phase 6: SQL Query Design

You must write 15 SQL queries.

They must include:

- selection
- projection
- joins
- aggregation
- grouping
- subqueries

Queries must represent real academic operations.

10. Phase 7: Query Processing Analysis

You must select 3 complex queries.

For each:

- execute query
- generate execution plan
- explain execution behavior

You must interpret how PostgreSQL processes the query, not just show screenshots.

11. Phase 8: Indexing Experiment

You must:

1. Run query without index
2. Record performance
3. Create index
4. Run again
5. Compare results

You must explain why performance changed.

12. Phase 9: Storage Investigation

You must analyze physical database storage:

- table size
- database size
- storage structure

You must explain:

- pages
- file organization
- how PostgreSQL stores data physically

13. Phase 10: Buffer Management

You must design an experiment showing:

- disk access before caching
- memory caching behavior
- performance improvement after repeated execution

You must clearly explain results.

IN BRIEF:

- Requirements Specification (with interview appendix)
- ER Diagram
- Enhanced ER Diagram
- PostgreSQL Schema
- SQL Scripts
- Sample Data
- 15 SQL Queries
- Query Analysis Report
- Indexing Report
- Storage Investigation Report
- Buffer Management Report
- Presentation Slides

Good work